EECS 348 Assignment 4: GenAI Code Analysis

Nicolas Villalobos

October 15, 2026

a. GenAI Programs Used b. How I Accessed Them

	GenAI 1	**GenAI 2**
Program (a)	Mistral AI, model Mistral Small 4 ("Fast" mode)	Anthropic Claude, model Claude Opus 5.5
Access (b)	Le Chat, Mistral's web chat app (chat.mistral.ai), in a new chat	claude.ai web chat, in a new chat with no earlier context
Output	One C++ file shown in Le Chat's canvas, plus a short design overview	One C++ file (`sudoku.cpp`) plus a short description of its classes and testing

c. Prompt Used (identical for both GenAIs)

Write a C++ program in a single source file that uses objects (classes), not standalone functions, to solve Sudoku puzzles. The solver must be recursive and use depth-first search with backtracking: find an empty cell, compute the list of candidate digits that do not conflict with its row, column, or 3x3 block, try each candidate and recurse, and backtrack if no candidate works. The program reads five files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. Each file has 9 lines with 9 space-separated entries; digits 1-9 are given values and an underscore (_) is a blank cell. For each file, print the file name, the puzzle as read from the file, and every solution to the puzzle (if there is more than one solution, print them all). If no solution exists, print "No solution found".

d. Generated Code

Both programs are reproduced below exactly as each GenAI produced them, with no edits.

GenAI 1: Mistral Small 4

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

// A Sudoku puzzle: holds the 9x9 grid, reads it from a file,
// prints it, and checks conflicts in a row, column, or 3x3 block.
class Sudoku {
private:
    static const int SIZE = 9;
    int grid[SIZE][SIZE];
    bool fixed[SIZE][SIZE];   // true if the value came from the file
    std::string fileName;

    // Is it legal to place 'digit' at (row, col)?
    // Returns false if digit conflicts with the row, column, or block.
    bool isLegal(int row, int col, int digit) const {
        for (int i = 0; i < SIZE; ++i) {
            if (grid[row][i] == digit) return false;  // row conflict
            if (grid[i][col] == digit) return false;  // column conflict
        }
        int blockRow = (row / 3) * 3;
        int blockCol = (col / 3) * 3;
        for (int r = blockRow; r < blockRow + 3; ++r)
            for (int c = blockCol; c < blockCol + 3; ++c)
```

```
                if (grid[r][c] == digit) return false; // block conflict
        return true;
    }

    // Build the list of candidate digits that do not conflict
    // with the row, column, or 3x3 block of cell (row, col).
    std::vector<int> candidates(int row, int col) const {
        std::vector<int> result;
        for (int digit = 1; digit <= 9; ++digit)
            if (isLegal(row, col, digit))
                result.push_back(digit);
        return result;
    }

public:
    // Load the puzzle from a file. Returns false if the file
    // cannot be opened or is malformed.
    bool load(const std::string& name) {
        fileName = name;
        std::ifstream in(name.c_str());
        if (!in) return false;

        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                std::string entry;
                if (!(in >> entry)) return false;
                if (entry == "_") {
                    grid[r][c] = 0;      // 0 means blank
                    fixed[r][c] = false;
                } else if (entry.length() == 1 && entry[0] >= '1' && entry[0] <= '9') {
                    grid[r][c] = entry[0] - '0';
                    fixed[r][c] = true;
                } else {
                    return false;       // invalid entry
                }
            }
        }
        return true;
    }

    // Print the puzzle exactly as read from the file:
    // digits for given values, underscores for blanks.
    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (c > 0) out << ' ';
                if (grid[r][c] == 0) out << '_';
                else                 out << grid[r][c];
            }
            out << '\n';
        }
    }

    const std::string& name() const { return fileName; }
    int at(int r, int c) const { return grid[r][c]; }
    void set(int r, int c, int v) { grid[r][c] = v; }
    bool isFixed(int r, int c) const { return fixed[r][c]; }
};

// Recursive depth-first solver with backtracking.
// Finds and prints EVERY solution of the puzzle.
class SudokuSolver {
private:
    Sudoku puzzle;
    int solutionCount;

    // Recursive backtracking search. Returns true if at least one
    // solution was found (so "No solution found" can be decided),
    // but keeps searching after success to find all solutions.
    bool solve() {
        // Find the first empty cell.
        int row = -1, col = -1;
        for (int r = 0; r < SIZE_ROW() && row == -1; ++r)
            for (int c = 0; c < SIZE_COL(); ++c)
                if (puzzle.at(r, c) == 0) { row = r; col = c; break; }

        // No empty cell left: the grid is a complete solution.
        if (row == -1) {
            ++solutionCount;
            puzzle.print(std::cout);
            std::cout << '\n';
            return true;
        }
```

```
109
110         // Try each candidate digit for this cell and recurse.
111         bool any = false;
112         std::vector<int> options = candidateList(row, col);
113         for (size_t i = 0; i < options.size(); ++i) {
114             puzzle.set(row, col, options[i]);
115             if (solve()) any = true;
116             puzzle.set(row, col, 0);   // backtrack
117         }
118         return any;
119     }
120
121     // Compute candidate digits for cell (row, col) by checking the
122     // row, column, and 3x3 block for conflicts.
123     std::vector<int> candidateList(int row, int col) {
124         std::vector<int> result;
125         for (int digit = 1; digit <= 9; ++digit)
126             if (isSafe(row, col, digit))
127                 result.push_back(digit);
128         return result;
129     }
130
131     // True if 'digit' does not appear in the row, column, or block
132     // containing (row, col).
133     bool isSafe(int row, int col, int digit) {
134         for (int i = 0; i < 9; ++i) {
135             if (puzzle.at(row, i) == digit) return false;
136             if (puzzle.at(i, col) == digit) return false;
137         }
138         int blockRow = (row / 3) * 3;
139         int blockCol = (col / 3) * 3;
140         for (int r = blockRow; r < blockRow + 3; ++r)
141             for (int c = blockCol; c < blockCol + 3; ++c)
142                 if (puzzle.at(r, c) == digit) return false;
143         return true;
144     }
145
146     // Small helpers so the solver does not depend on a constant
147     // defined inside the Sudoku class.
148     static int SIZE_ROW() { return 9; }
149     static int SIZE_COL() { return 9; }
150
151 public:
152     // Load a puzzle file, print it, solve it, and print every solution.
153     bool processFile(const std::string& fileName) {
154         if (!puzzle.load(fileName)) {
155             std::cout << fileName << '\n';
156             std::cout << "Could not read puzzle file.\n\n";
157             return false;
158         }
159
160         std::cout << fileName << '\n';
161         std::cout << "Puzzle:\n";
162         puzzle.print(std::cout);
163         std::cout << "Solutions:\n";
164
165         solutionCount = 0;
166         bool found = solve();
167         if (!found)
168             std::cout << "No solution found\n";
169         else if (solutionCount > 1)
170             std::cout << '(' << solutionCount << " solutions)\n";
171         std::cout << '\n';
172         return true;
173     }
174 };
175
176 // Application object: drives the whole program, so the logic
177 // lives in objects rather than standalone functions.
178 class SudokuApp {
179 private:
180     std::vector<std::string> fileNames;
181
182     void run() {
183         const char* defaults[5] = {
184             "puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"
185         };
186         for (int i = 0; i < 5; ++i)
187             fileNames.push_back(defaults[i]);
188
189         for (size_t i = 0; i < fileNames.size(); ++i) {
190             SudokuSolver solver;
191             solver.processFile(fileNames[i]);
```

```
        }
    }

public:
    int execute() {
        run();
        return 0;
    }
};

int main() {
    SudokuApp app;
    return app.execute();
}
```

GenAI 2: Claude Opus 5.5

```
// sudoku.cpp
// Recursive depth-first search Sudoku solver with backtracking.
// Reads puzzle1.txt .. puzzle5.txt and prints every solution for each puzzle.

#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// SudokuBoard: stores a 9x9 grid (0 = blank) and knows the Sudoku rules.
// ---------------------------------------------------------------------------
class SudokuBoard {
public:
    static const int SIZE = 9;
    static const int BOX = 3;
    static const int EMPTY = 0;

    SudokuBoard() {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c)
                cells[r][c] = EMPTY;
    }

    // Load from a file of 9 lines, 9 space-separated tokens each.
    // Tokens are digits 1-9 or '_' for a blank. Returns false on any error.
    bool loadFromFile(const std::string& fileName, std::string& error) {
        std::ifstream in(fileName);
        if (!in) {
            error = "Could not open file";
            return false;
        }
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                std::string token;
                if (!(in >> token)) {
                    error = "File ended early (expected 81 entries)";
                    return false;
                }
                if (token == "_") {
                    cells[r][c] = EMPTY;
                } else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') {
                    cells[r][c] = token[0] - '0';
                } else {
                    error = "Invalid entry '" + token + "' at row " +
                            std::to_string(r + 1) + ", column " + std::to_string(c + 1);
                    return false;
                }
            }
        }
        return true;
    }

    int get(int row, int col) const { return cells[row][col]; }
    void set(int row, int col, int value) { cells[row][col] = value; }

    // Find the next empty cell in row-major order. Returns false if the board is full.
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < SIZE; ++row)
            for (col = 0; col < SIZE; ++col)
                if (cells[row][col] == EMPTY)
                    return true;
        return false;
```

```
    }

    // True if placing value at (row, col) does not conflict with the
    // row, column, or 3x3 block (the cell itself is ignored).
    bool canPlace(int row, int col, int value) const {
        for (int i = 0; i < SIZE; ++i) {
            if (i != col && cells[row][i] == value) return false;
            if (i != row && cells[i][col] == value) return false;
        }
        int boxRow = (row / BOX) * BOX;
        int boxCol = (col / BOX) * BOX;
        for (int r = boxRow; r < boxRow + BOX; ++r)
            for (int c = boxCol; c < boxCol + BOX; ++c)
                if ((r != row || c != col) && cells[r][c] == value)
                    return false;
        return true;
    }

    // List of digits that can legally go in (row, col).
    std::vector<int> candidates(int row, int col) const {
        std::vector<int> result;
        for (int value = 1; value <= SIZE; ++value)
            if (canPlace(row, col, value))
                result.push_back(value);
        return result;
    }

    // Check that the given clues do not already conflict with each other.
    bool givensAreConsistent() const {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c)
                if (cells[r][c] != EMPTY && !canPlace(r, c, cells[r][c]))
                    return false;
        return true;
    }

    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            if (r > 0 && r % BOX == 0)
                out << "------+-------+------\n";
            for (int c = 0; c < SIZE; ++c) {
                if (c > 0 && c % BOX == 0) out << "| ";
                if (cells[r][c] == EMPTY) out << '_';
                else out << cells[r][c];
                if (c < SIZE - 1) out << ' ';
            }
            out << '\n';
        }
    }

private:
    int cells[SIZE][SIZE];
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive DFS with backtracking that collects ALL solutions.
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    explicit SudokuSolver(const SudokuBoard& start) : board(start) {}

    // Runs the search and returns every solution found.
    const std::vector<SudokuBoard>& solveAll() {
        solutions.clear();
        if (board.givensAreConsistent())
            search();
        return solutions;
    }

private:
    SudokuBoard board;                    // working copy, modified during search
    std::vector<SudokuBoard> solutions;   // every completed grid

    void search() {
        int row, col;
        // Base case: no empty cell left means the board is a valid solution.
        if (!board.findEmptyCell(row, col)) {
            solutions.push_back(board);
            return;
        }
        // Try each non-conflicting digit, recurse, then undo (backtrack).
        std::vector<int> options = board.candidates(row, col);
        for (int value : options) {
```

```
            board.set(row, col, value);
            search();
            board.set(row, col, SudokuBoard::EMPTY);
        }
        // If options was empty, or every branch dead-ended, we simply return
        // and the caller tries its next candidate.
    }
};

// ---------------------------------------------------------------------------
// PuzzleRunner: handles one file end to end (read, print, solve, report).
// ---------------------------------------------------------------------------
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string& file) : fileName(file) {}

    void run(std::ostream& out) const {
        out << "============================\n";
        out << "File: " << fileName << "\n";
        out << "============================\n";

        SudokuBoard puzzle;
        std::string error;
        if (!puzzle.loadFromFile(fileName, error)) {
            out << "Error reading " << fileName << ": " << error << "\n\n";
            return;
        }

        out << "Puzzle:\n";
        puzzle.print(out);
        out << '\n';

        SudokuSolver solver(puzzle);
        const std::vector<SudokuBoard>& solutions = solver.solveAll();

        if (solutions.empty()) {
            out << "No solution found\n\n";
            return;
        }

        out << solutions.size()
            << (solutions.size() == 1 ? " solution found:\n" : " solutions found:\n");
        for (std::size_t i = 0; i < solutions.size(); ++i) {
            out << "\nSolution " << (i + 1) << ":\n";
            solutions[i].print(out);
        }
        out << '\n';
    }

private:
    std::string fileName;
};

// ---------------------------------------------------------------------------
// SudokuApp: drives the five puzzle files.
// ---------------------------------------------------------------------------
class SudokuApp {
public:
    SudokuApp() {
        for (int i = 1; i <= 5; ++i)
            files.push_back("puzzle" + std::to_string(i) + ".txt");
    }

    void run(std::ostream& out) const {
        for (const std::string& file : files) {
            PuzzleRunner runner(file);
            runner.run(out);
        }
    }

private:
    std::vector<std::string> files;
};

int main() {
    SudokuApp app;
    app.run(std::cout);
    return 0;
}
```

e. Code Analysis

Test setup. Both programs were compiled with `g++ -std=c++11 -O2 -Wall -Wextra` (g++ 13.3, Linux x86-64) and run on the five assignment puzzle files. Timings are the average of 3,000 in-process runs per puzzle (3 runs for the two stress tests), with output sent to `/dev/null` so printing speed does not dominate. Peak memory is the maximum resident set size of the whole process. Besides the five given files I used two stress tests: **hard.txt**, a well known 21-clue puzzle designed to defeat simple backtracking, and **sparse.txt**, puzzle1 with its bottom three rows blanked, which has 29,040 solutions.

Correctness

Test	Mistral Small 4	Claude Opus 5.5
puzzle1.txt (vs. solution1.txt)	1 solution, correct	1 solution, correct
puzzle2.txt, puzzle3.txt	1 solution each, correct	1 solution each, correct
puzzle4.txt	No solution found (correct)	No solution found (correct)
puzzle5.txt (2 solutions)	Both printed (correct)	Both printed (correct)
Conflicting givens (solution1 with cell (1,2) changed to a second 5, no blanks)	**Wrong:** prints the invalid grid as a solution, because it only checks digits it places, never the givens	Correct: `givensAreConsistent()` rejects it and prints *No solution found*
Malformed file (bad token, short file)	Detected, but only prints "Could not read puzzle file."	Detected, and reports the reason plus the row and column of the bad entry
Output labeling	Solutions not numbered; a count is printed only when there are 2 or more	Every solution numbered; count always printed

Both programs implement the required recursive depth-first search with backtracking, and both are object-oriented: all logic is inside classes and `main()` only creates an application object. On the five provided files they produce identical solution grids. The difference is robustness. Mistral's solver can report an invalid grid as a valid solution, which is a real correctness bug and not a style issue.

Execution Time

Input	Mistral (μs)	Claude (μs)	Final (μs)	Calls: Claude / Final
puzzle1.txt	19.5	25.5	9.8	167 / 40
puzzle2.txt	10.6	14.0	10.8	31 / 29
puzzle3.txt	13.8	16.1	9.3	57 / 33
puzzle4.txt	6.6	9.0	6.0	17 / 13
puzzle5.txt	103.5	154.4	14.0	1,269 / 61
hard.txt	225,379	270,434	3,602	2,068,781 / 24,881
sparse.txt (29,040 sol.)	217,787	301,703	107,331	1,141,140 / 415,906

The *Final* column is my improved program (section g), included so all numbers can be compared in one place. "Calls" counts recursive `search()` calls.

Asymptotically the two GenAI programs run the same algorithm. With k blank cells the search tree has at most 9^k leaves, so the worst case is

$$T(k) = O\big(9^k\big)\,.$$

The cost per recursive call is also the same order for both: an $O(81)$ scan from the top-left to find the first empty cell, then 9 legality checks of 27 cells each ($9 \times 27 = 243$ comparisons), plus a heap-allocated `std::vector` for the candidate list. Mistral is about 20–30% faster in practice, most likely because Claude's `canPlace()` adds an extra "is this the same cell" comparison inside every loop iteration (it needs that to reuse the function for validating givens). Both always choose the *first* empty cell in row-major order, which is why hard.txt needs over 2 million calls.

Space Complexity

	Mistral	Claude	Final
Grid storage	`int[9][9]` + `bool[9][9]` (bool array never used)	`int[9][9]`	`int[9][9]` + 27 int masks
Recursion stack	$O(k)$ frames, each with a candidate vector	$O(k)$ frames, each with a candidate vector	$O(k)$ frames, no heap allocation
Solutions	Printed immediately: $O(1)$	All stored in `vector<SudokuBoard>`: $O(81S)$ for S solutions	Printed immediately: $O(1)$
Peak memory, 5 given puzzles	3,728 KB	3,684 KB	3,644 KB
Peak memory, sparse.txt	3,704 KB	**13,792 KB**	3,644 KB

On the given puzzles, memory is dominated by the C++ runtime and all three programs are the same. Claude's choice to store every solution before printing grows linearly with the number of solutions:

$$29{,}040 \text{ boards} \times 324 \text{ bytes} \approx 9.2 \text{ MB},$$

which matches the extra ∼10 MB measured. Mistral streams solutions and stays flat.

Maintainability

Mistral has noticeable problems:

- Dead code: `Sudoku::isLegal()`, `Sudoku::candidates()`, the `fixed[][]` array, `isFixed()` and `name()` are never used.
- Duplicated logic: `SudokuSolver::isSafe()` and `candidateList()` re-implement rules that already exist in the `Sudoku` class.
- Magic numbers: because `Sudoku::SIZE` is private, the solver works around it with `SIZE_ROW()`/`SIZE_COL()` helpers that return a hard-coded 9.
- Hard to test: `solve()` writes straight to `std::cout` instead of a stream passed in.
- Fragile driver: `SudokuApp::run()` appends the file names every time it runs, so calling `execute()` twice would process every file twice.
- Const-correctness: `candidateList()` and `isSafe()` only read state but are not `const`.

Claude is cleaner. Each class has one job (`SudokuBoard` holds the grid, rules and I/O; `SudokuSolver` searches; `PuzzleRunner` handles one file; `SudokuApp` drives all five). It uses named constants (`SIZE`, `BOX`, `EMPTY`) instead of literals, has no dead code, marks read-only methods `const`, and writes output to a `std::ostream` parameter so it can be redirected. Its weak points are the per-call vector allocation and the stored-solutions design.

Summary

Metric	Mistral Small 4	Claude Opus 5.5
Correctness	Passes the 5 files; fails on conflicting givens	**Passes every test**
Execution time	**About 20–30% faster**	Slower, same complexity
Space	$O(1)$ **for solutions**	$O(81S)$ for solutions
Maintainability	Dead code, duplicated rules, magic numbers	**Clean separation, constants, const methods**

f. Selected Baseline and Justification

I selected **Claude Opus 5.5** as the baseline.

- **Correctness comes first.** Claude passed every test, including conflicting givens, where Mistral printed an invalid grid as a solution. A wrong answer is worse than a slow one.
- **Its weaknesses are cheap to fix; Mistral's are not.** Claude's losses on time (20–30%) and space (stored solutions) come from two localized choices inside `SudokuSolver`. Fixing Mistral would mean adding validation, deleting dead code, removing duplicated rules and restructuring the classes, which touches nearly every part of the file.
- **Its structure supports the changes I wanted.** Because the search lives entirely in `SudokuSolver` and output goes through a stream parameter, I could replace the search internals without touching file loading, printing, or the driver.

g. Improvements Made to the Baseline

Change	Metric	Effect (measured)
Track used digits with three bitmask arrays (`rowUsed`, `colUsed`, `boxUsed`), updated by `place()` and `remove()`	Execution time, space	Checking whether a digit is legal drops from scanning 27 cells to one OR and one AND. All candidates for a cell come from one expression, so no per-call `std::vector` is allocated.
Pick the empty cell with the fewest candidates instead of the first one, and stop immediately if any empty cell has zero candidates	Execution time	Far fewer recursive calls: 2,068,781 → 24,881 on hard.txt (75× faster, 270 ms → 3.6 ms) and 1,269 → 61 on puzzle5.txt (11× faster). Still exhaustive, so every solution is still found.
Print each solution the moment it is found instead of storing all of them	Space	Solution storage goes from $O(81S)$ to $O(1)$. Peak memory on sparse.txt drops from 13,792 KB to 3,644 KB.
Check the givens for conflicts in the same single pass that builds the bitmasks	Correctness, time	Keeps the baseline's conflicting-givens protection using one pass over 81 cells instead of 81×27 comparisons. Verified: the conflicting-givens test still prints *No solution found.*
Print the puzzle in the same layout as the input file and end each file with "Solutions found: N"	Correctness (requirements)	The assignment asks for the puzzle from the file printed out; the baseline added `|` and `---` separators. The count line matches solution1.txt. Final puzzle1 output matches solution1.txt exactly.
Moved rule checks out of `SudokuBoard` so the rules live in one place, split cell choice into `chooseCell()`, and verified C++11 compatibility	Maintainability	One source of truth for the rules, a shorter `search()`, and a clean build with `-std=c++11 -Wall -Wextra -pedantic`, so it compiles on the cycle servers without needing C++17.
Added the required prologue, block headers stating who wrote each block, and comments on every line	Maintainability	Every block is attributed (Claude baseline, my change, or both) as the course requires.

Result: the final program produces the same solutions as both GenAI programs on all five puzzle files (puzzle1: 1, puzzle2: 1, puzzle3: 1, puzzle4: none, puzzle5: 2), keeps Claude's correctness on bad input, is faster than Mistral on every puzzle, and matches Mistral's constant memory for solutions.